# Московский авиационный институт
# (национальный исследовательский университет)

## Факультет информационных технологий и прикладной математики

## Кафедра вычислительной математики и программирования


**Лабораторная работа №1 по курсу «Дискретный анализ»**


| | |
|---|---|
| Студент: | Р. А. Савицкас |
| Преподаватель: | А. А. Кухтичев |
| Группа: | М8О-206БВ-24 |
| Дата: | |
| Оценка: | |
| Подпись: | |



**Москва, 2026**

# Лабораторная работа №1

## Задание

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа за линейное время и вывод отсортированной последовательности.

Согласно заданию, необходимо реализовать собственный шаблонный класс для управления динамической памятью (аналог `std::vector`).

**Вариант алгоритма:** Поразрядная сортировка (Radix Sort).

**Вариант ключа:** `Автомобильные номера в формате A 999 BC (используются буквы латинско алфавита).`

**Вариант значения:** `Строки переменной длины (до 2048 символов).`

**Ограничение:** `Использование стандартных контейнеров (STL) запрещено. Сортировка должна выполняться за время` $O(N)$.

# 1 Исходный код

На каждой непустой строке входного файла располагается пара «ключ-значение», в которой ключ указан согласно заданию, затем следует знак табуляции и указано соответствующее значение.

Что касается самой поразрядной сортировки, как сказано в [3]: «основная идея заключается в том, что сортировка выполняется поразрядно, от младших разрядов к старшим (LSD) или наоборот (MSD). При этом на каждом шаге должна использоваться устойчивая сортировка, чтобы сохранить относительный порядок элементов с одинаковыми значениями в текущем разряде».

Для реализации алгоритма была создана структура `Pair` для хранения данных и шаблонный класс `MyVector` для управления динамическим массивом. Процесс сортировки разделен на этапы: функция `get_digit` извлекает разряды ключа, которые затем обрабатываются в функции `radix_sort` с применением сортировки подсчётом.

```
#include <iostream>
#include <string>
#include <utility>

template <typename T>
class MyVector {
private:
    T* data;
    size_t sz;
    size_t capacity;

    void reserve(size_t new_capacity) {
        if (new_capacity <= capacity) return;
        T* new_data = new T[new_capacity];
        for (size_t i = 0; i < sz; ++i) {
            new_data[i] = std::move(data[i]);
        }
        delete[] data;
        data = new_data;
        capacity = new_capacity;
    }

public:
    MyVector() : data(nullptr), sz(0), capacity(0) {}
    MyVector(size_t n) : data(new T[n]), sz(n), capacity(n) {}
    ~MyVector() { delete[] data; }

    MyVector(const MyVector& other) : data(nullptr), sz(0), capacity(0)
    ↪ {
        reserve(other.capacity);
        for (size_t i = 0; i < other.sz; ++i) {
            data[i] = other.data[i];
        }
        sz = other.sz;
    }

    MyVector& operator=(MyVector other) {
        std::swap(data, other.data);
```

```
        std::swap(sz, other.sz);
        std::swap(capacity, other.capacity);
        return *this;
    }

    void push_back(T&& value) {
        if (sz == capacity) reserve(capacity == 0 ? 1 : capacity * 2);
        data[sz++] = std::move(value);
    }

    T& operator[](size_t i) { return data[i]; }
    const T& operator[](size_t i) const { return data[i]; }
    size_t size() const { return sz; }
};

struct Pair {
    char plate[9];
    std::string value;
};

int get_digit(const Pair& p, int step) {
    switch (step) {
        case 0: return p.plate[7] - 'A';
        case 1: return p.plate[6] - 'A';
        case 2:
            return (p.plate[2] - '0') * 100 + (p.plate[3] - '0') * 10 + (p.plate[4] - '0');
        case 3: return p.plate[0] - 'A';
        default: return 0;
    }
}

void radix_sort(MyVector<Pair>& vec) {
    size_t n = vec.size();
    if (n < 2) return;

    Pair* temp = new Pair[n];
    Pair* src = &vec[0];
    Pair* dst = temp;

    for (int step = 0; step < 4; ++step) {
        int range = (step == 2) ? 1000 : 26;
        int count[1000] = {0};

        for (size_t i = 0; i < n; ++i) count[get_digit(src[i], step)]++;
        for (int i = 1; i < range; ++i) count[i] += count[i - 1];

        for (int i = (int)n - 1; i >= 0; --i) {
            int d = get_digit(src[i], step);
            dst[--count[d]] = std::move(src[i]);
        }
        std::swap(src, dst);
```

```
    }

    if (src != &vec[0]) {
        for (size_t i = 0; i < n; ++i) vec[i] = std::move(src[i]);
    }
    delete[] temp;
}

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    MyVector<Pair> data;
    char p1;
    int num;
    std::string p_last;

    while (std::cin >> p1 >> num >> p_last) {
        Pair p;
        p.plate[0] = p1; p.plate[1] = ' ';
        p.plate[2] = (num / 100) + '0';
        p.plate[3] = ((num / 10) % 10) + '0';
        p.plate[4] = (num % 10) + '0';
        p.plate[5] = ' ';
        p.plate[6] = p_last[0];
        p.plate[7] = p_last[1];
        p.plate[8] = '\0';

        std::string val;
        std::getline(std::cin, val);
        if (!val.empty() && val[0] == '\t') p.value = val.substr(1);
        else p.value = val;

        data.push_back(std::move(p));
    }

    radix_sort(data);

    for (size_t i = 0; i < data.size(); ++i) {
        std::cout << data[i].plate << "\t" << data[i].value << "\n";
    }
    return 0;
}
```

## 2 Тест производительности

В данном разделе проводится анализ эффективности реализованного алгоритма поразрядной сортировки в сравнении со стандартной библиотечной функцией `std::stable_sort` (реализация Merge Sort).

Тест производительности проводился на случайно сгенерированных данных. Объем входной последовательности составил 500 000 пар «ключ-значение», где ключи представляют собой уникальные и повторяющиеся государственные номера, а значения — строки длиной до 2048 символов. Время замерялось с помощью библиотеки `chrono`.

```
ame@ame:~/Discrete-Analysis/lab1$ make benchmark
g++ -O3 -Wall -Wextra -std=c++17 -c src/benchmark.cpp -o benchmark.o
g++ -O3 -Wall -Wextra -std=c++17 -c src/sort_logic.cpp -o sort_logic.o
g++ -O3 -Wall -Wextra -std=c++17 benchmark.o sort_logic.o -o benchmark
ame@ame:~/Discrete-Analysis/lab1$ python3 py/gen.py 100000 > input.txt
ame@ame:~/Discrete-Analysis/lab1$ ./benchmark < input.txt
Amount of lines: 100000
-----------------------------------
My Radix Sort:  4377 us
Stable Sort:    22112 us
-----------------------------------
Result: My Radix Sort is 5.05x faster than Stable Sort.
```

Как видно из результатов, поразрядная сортировка выигрывает у стандартного алгоритма более чем в 5 раза. Это обусловлено тем, что `Radix Sort` работает за линейное время $O(N)$ и не выполняет операцию лексикографического сравнения строк целиком, ограничиваясь диапазоном значений в данном разряде. В то время как `stable_sort` имеет сложность $O(N \log N)$ и на каждой итерации вынужден обращаться к строковым данным для сравнения ключей.

Также стоит отметить, что использование семантики перемещения (`std::move`) в кастомном векторе позволило избежать лишних аллокаций памяти при сортировке тяжелых строковых значений.

# 3 Выводы

*Выполнив первую лабораторную работу по курсу «Дискретный анализ», я закрепил навыки проектирования эффективных систем обработки данных на языке C++ и освоил применение алгоритмов с линейной сложностью.*

В ходе работы я пришел к следующим выводам:

- **Эффективность специализированных алгоритмов:** Реализованный алгоритм *Radix Sort* подтвердил свою эффективность на практике. Благодаря тому, что структура ключа (автомобильного номера) фиксирована, удалось достичь стабильного ускорения примерно в 5 раза по сравнению с `std::stable_sort`. Это доказывает, что знание природы данных позволяет выбирать более оптимальные инструменты, чем универсальные решения.

- **Управление ресурсами:** Реализация собственного шаблона `MyVector` позволила глубже понять механизмы работы с динамической памятью. Особое внимание было уделено семантике перемещения (`T&&` и `std::move`). Я убедился, что для «тяжелых» объектов, таких как строки `std::string`, перемещение указателей вместо копирования буферов является критически важным фактором производительности.

- **Низкоуровневая оптимизация:** Я узнал, как значительно ускорить ввод-вывод в C++ через отключение синхронизации с `stdio`. В условиях обработки сотен тысяч строк разница в скорости парсинга становится ощутимой.

Данный опыт будет полезен в будущих задачах, где требуется обработка больших потоков информации в реальном времени, а также при написании высокопроизводительных системных компонентов.

# Список литературы


[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 3-е издание.* — М.: ООО «И.Д. Вильямс», 2013. — 1328 с. (ISBN 978-5-8459-1794-2)

[2] *Системы ввода-вывода в C++. Оптимизация потоков std::cin и std::cout.* URL: `https://en.cppreference.com/w/cpp/io/ios_base/sync_with_stdio` (дата обращения: 03.03.2026).

[3] *Поразрядная сортировка (Radix sort) — Википедия.* URL: `https://ru.wikipedia.org/wiki/Поразрядная_сортировка` (дата обращения: 03.03.2026).